

Y-Dude — Production Load Test bis 750 virtuelle Nutzer

Messbericht · Production <https://y-dude.com> · 30.08.2026, 04:20-05:05 UTC · read-only, ausschließlich anonyme synthetische Sessions

1. Executive Summary

Die veröffentlichte Production-Version wurde in fünf Stufen (50 / 100 / 250 / 500 / 750 virtuelle Nutzer) mit realistischen Lesepfaden belastet. Alle fünf Stufen wurden erreicht und blieben stabil. Über 94.720 Requests traten 0 Serverfehler (5xx), 0 4xx-Fehler und 1 Timeout auf. Die Antwortzeiten blieben über alle Stufen nahezu konstant (p95 45–66 ms); bei 750 Nutzern lag der Durchsatz bei 366 Requests/s mit p50 32 ms, p95 45 ms, p99 114 ms.

750 virtuelle Nutzer erfolgreich getestet. Ein Bottleneck wurde im getesteten Lastbereich nicht erreicht — weder Backend, Datenbank, Auth, Storage noch CDN zeigten Sättigungsanzeichen. Der SSR-Kurzzeitcache für öffentliche Beitragsseiten arbeitete unter Last mit steigender Trefferquote (79 % bei 50 VUs → 92 % bei 750 VUs) und war damit der wirksamste Skalierungshebel der Messung. Einschränkung der Messung: Angemeldete Pfade (Feed-Daten, View-Batch, Translation-Batch, Messenger, Market-Daten) konnten nicht belastet werden, weil keine dedizierten Testkonten in Production existieren und laut Auftrag keine Testdaten angelegt werden durften. Serverseitige Kennzahlen (CPU, RAM, DB-Connections) sind nicht messbar: der interne Messendpunkt antwortet in Production mit HTTP 500, der DB-Health-Abruf mit „metrics payload exceeded size cap“. Diese beiden Punkte wurden auftragsgemäß NICHT behoben.

2. Testziel

Ermitteln, wie sich die aktuell veröffentlichte Production-Version unter steigender gleichzeitiger synthetischer Nutzerlast verhält, wo Latenz und Fehlerquote messbar ansteigen und wo die tatsächlichen Skalierungsgrenzen liegen. Reiner Mess- und Analyseauftrag: keine Optimierung, keine Code-, DB-, RLS-, Index-, CDN-, Storage- oder Secret-Änderung.

3. Production-Ausgangszustand (gemessen vor der Last)

Prüfpunkt	Ergebnis
Kanonischer Host	y-dude.com (y-dude.lovable.app und www.y-dude.com → 302 auf y-dude.com)
Erreichbarkeit /, /auth, /agb, /market, /globe, /richtlinien	alle HTTP 200
Öffentliche Beitragsseite /post/<uuid>	HTTP 200, Header vary: Cookie, Authorization, x-ydude-cache vorhanden
cache-control auf HTML	no-cache, must-revalidate, max-age=0 (der s-maxage-Wert des Performance-Release greift nicht, da bereits eine cache-control-Kopfzeile gesetzt ist)
Auth-Endpunkt (GoTrue health)	HTTP 200, 90 ms
Anonymer Lesezugriff posts	HTTP 401 permission denied (erwartet)
Anonymer Lesezugriff comments	HTTP 200 (bekannte, zuvor als Rating C dokumentierte Altlast)
Interner Metrik-Endpunkt /api/public/cache-metrics	HTTP 500 nach erfolgreicher Autorisierung → Serverkennzahlen nicht messbar
DB-Health-Snapshot	Fehler „metrics payload exceeded size cap“ → DB-CPU/Connections nicht messbar

Feststellung (Messung, keine Änderung): /feed und /dashboard existieren in Production nicht (404); der Feed-Einstieg läuft über /arena (307 auf das Login-Gate für Anonyme).

4. Testmethodik

Eigener Lastgenerator (Bun/TypeScript, Sandbox in Frankreich, CF-Edge CDG). Jeder virtuelle Nutzer durchläuft in Endlosschleife eine realistische Sitzung mit Wartezeiten von 0,5–4 s zwischen den Aktionen:

1. Landing / — 2. Bundle-Asset (CDN) — 3. Login-Seite /auth — 4. Auth-API (GoTrue health) — 5. Feed-Einstieg /arena — 6. Marketplace /market — 7. Globe /globe — 8. zwei bis drei öffentliche Beiträge /post/<uuid> (12 reale, öffentliche Beiträge, zufällig gewählt) — 9. Kommentare des jeweiligen Beitrags (Datenpfad mit RLS) — 10. eine Rechtsseite (AGB/Datenschutz/Impressum/Richtlinien).
Ramp-up 10 s je Stufe, Messdauer 90 s (Stufe 750: 120 s), zwischen den Stufen Stabilisierungspause. Timeout 15 s. Automatischer Abbruch, sobald über 10 % der letzten 300 Requests mit 5xx oder Timeout enden. Keine Schreibvorgänge, keine Logins, keine Zahlungen, keine E-Mails, keine Testdaten, keine echten Nutzerkonten. Redirects wurden nicht gefolgt (307 des Login-Gates wird als Erfolg gezählt).

5.-9. Laststufen und Messwerte

VUs	Dauer	Requests	Req/s	Erfolg	Fehler	5xx	4xx	Timeouts	p50	p95	p99	max
50	103 s	2490	24	2490	0	0	0	0	33 ms	66 ms	150 ms	470 ms
100	104 s	4948	48	4948	0	0	0	0	35 ms	51 ms	134 ms	761 ms
250	105 s	12432	118	12432	0	0	0	0	34 ms	53 ms	139 ms	690 ms
500	106 s	24898	235	24898	0	0	0	0	32 ms	45 ms	118 ms	1109 ms
750	136 s	49952	366	49951	1	0	0	1	32 ms	45 ms	114 ms	15001 ms

Latenz je Nutzeraktion (p95 / p99 in ms)

Aktion	50 VUs	100 VUs	250 VUs	500 VUs	750 VUs
Login-Seite	42 / 69	44 / 246	46 / 66	40 / 53	40 / 62
Auth-API	48 / 81	47 / 73	44 / 66	43 / 54	43 / 75
Feed-Einstieg	36 / 38	41 / 64	41 / 72	35 / 53	36 / 49
öffentl. Post (SSR)	151 / 196	125 / 159	131 / 258	115 / 144	109 / 139
Kommentare (RLS)	49 / 77	48 / 58	47 / 77	48 / 104	47 / 80
Marketplace	45 / 56	48 / 160	51 / 81	46 / 71	46 / 69
Globe	44 / 53	48 / 254	50 / 77	46 / 69	44 / 60
Asset (CDN)	32 / 61	37 / 62	37 / 112	32 / 39	32 / 45
Landing	39 / 117	44 / 96	50 / 257	41 / 60	42 / 90
Rechtsseiten	42 / 56	50 / 69	45 / 68	41 / 52	41 / 52

Alle Stufen: 5xx = 0, 4xx = 0. Der einzige Timeout der gesamten Messung trat in Stufe 750 auf (1 von 49.952 Requests = 0,002 %) und erklärt dort den Maximalwert von 15.001 ms (Abbruch durch das Client-Timeout).

10. Datenbank- und Backend-Auslastung

Kennzahl	Ergebnis
DB CPU	nicht messbar (DB-Health-Abfrage schlägt mit „metrics payload exceeded size cap“ fehl)
DB Connections / Connection Pool	nicht messbar (gleiche Ursache)
Query-Latenzen / RLS-Query-Latenzen (serverseitig)	nicht messbar; indirekt über Endpunktlatenz: Kommentare (RLS-Pfad) p95 49 ms bei 50 VUs, 46 ms bei 750 VUs — kein Anstieg
Server CPU / RAM / Event-Loop-Lag	nicht messbar (interner Metrik-Endpoint antwortet mit HTTP 500)
Storage Requests	nicht messbar; im Testpfad wurden keine Medien-Originale abgerufen (nur HTML + ein Bundle-Asset)
Netzwerk/Bandbreite	nicht direkt messbar; Lastgenerator-Durchsatz max. ca. 366 Requests/s, überwiegend HTML/JSON
View-Batch-Requests	nicht messbar — nur im angemeldeten Feed aktiv, mangels Testkonto nicht belastet
Translation-Batch-Requests	nicht messbar — gleiche Ursache

11. Cache-Verhalten (gemessen)

VUs	Post-Seiten	SSR HIT	SSR MISS	HIT-Quote	p50 HTML	p95 Post	p99 Post
50	437	345	92	79 %	33 ms	151 ms	196 ms
100	858	748	110	87 %	35 ms	125 ms	159 ms
250	2172	1881	291	87 %	34 ms	131 ms	258 ms
500	4361	3937	424	90 %	32 ms	115 ms	144 ms
750	9083	8340	743	92 %	32 ms	109 ms	139 ms

Auch Landing (192/202 HIT bei 50 VUs), Login-Seite (197/202) und Rechtsseiten (183/202) wurden aus dem SSR-Kurzzeitcache bedient. Die HIT-Quote steigt mit der Last, weil mehr Anfragen innerhalb der TTL auf dieselbe Worker-Instanz treffen. Bei Einzelaufrufen (Leerlauf, sequenziell) wurde dagegen mehrfach „miss“ gemessen: der Cache ist instanzlokal, nicht geteilt. Zusätzlich trägt das HTML weiterhin cache-control: no-cache — der im Performance-Release vorgesehene s-maxage/CDN-Anteil wird dadurch in Production nicht wirksam (Messung, keine Änderung).

12. View-/Translation-Batch-Verhalten

Nicht messbar. Beide Mechanismen greifen ausschließlich im angemeldeten Feed. Für Production existieren keine dedizierten Testkonten, und das Anlegen von Testdaten war ausdrücklich untersagt. Die Launchpad-Referenzwerte (View 6 → 1 Request, Translation 3-4 → 1 Request, 26-27 → 18-19 Backend-Requests bei 6 Posts) konnten daher nicht gegen Production verifiziert werden. Vergleichbar ist nur der SSR-HIT: Referenz ca. 4-5 ms lokal, in Production liefert eine gecachte Beitragsseite Ende-zu-Ende p50 ≈ 31 ms inklusive Internet-Roundtrip zum Edge.

13. Sicherheitsprüfung unter/nach Last

Prüfung	Ergebnis
SSR-Cache: öffentliche Posts HIT/MISS korrekt	OK — nur wirklich öffentliche Beiträge werden gecacht; eine unbekannte/nicht öffentliche UUID erzeugt keinen Cache-Eintrag und keine Cache-Kopfzeile
Cookie/Authorization → kein gemeinsamer Cache	OK — Anfragen mit Cookie oder Authorization erhalten weder x-ydude-cache noch den internen Marker; vary: Cookie, Authorization ist gesetzt
Interner Marker x-ydude-public-post nie ausgeliefert	OK — auf allen geprüften Pfaden entfernt
RLS unter Last	OK — anonym: posts 401 permission denied, profiles 401, messages leere Menge; keine Umgehung beobachtet

Prüfung	Ergebnis
Session-Isolation	OK — keine Cross-User-Daten; der Lasttest lief vollständig ohne Sitzung, ausgelieferte Seiten waren identisch und unpersonalisiert
React Query Focus-Refetch	nicht messbar auf HTTP-Ebene (Browser-Verhalten); Code-Default refetchOnWindowFocus: false unverändert aktiv
Beobachtung (unverändert)	comments ist anonym lesbar (HTTP 200, Inhalte). Entspricht dem bereits dokumentierten Audit-Rating C, wurde auftragsgemäß nicht geändert

14. Erster erkennbarer Bottleneck

Im getesteten Bereich bis 750 virtuelle Nutzer wurde kein Bottleneck erreicht. Latenz und Fehlerquote steigen nicht mit der Last — p95 sinkt von 66 ms (50 VUs) auf 45 ms (750 VUs), weil die Cache-Trefferquote steigt. Der erste erkennbare Engpass ist damit nicht serverseitig, sondern messtechnisch: die fehlende Beobachtbarkeit (Metrik-Endpoint 500, DB-Health nicht abrufbar) verhindert die Bestimmung der Sättigungsgrenze.

15. Skalierungsbewertung (nur aus Messwerten)

Frage	Antwort aus der Messung
Bis zu welcher Nutzerzahl stabil?	mindestens 750 gleichzeitige virtuelle Leser, 366 Req/s, 0 5xx
Wo steigt Latenz messbar?	nirgends im getesteten Bereich (p95 45-66 ms, ohne Trend nach oben)
Wo steigt die Fehlerquote?	nirgends; 1 Timeout bei 750 VUs = 0,002 %
Erster Bottleneck	im Messbereich nicht erreicht
Datenbank Bottleneck?	nein, soweit sichtbar — RLS-Lesepfad (Kommentare) bleibt konstant bei p95 ≈ 46-49 ms. Direkte DB-Kennzahlen: nicht messbar
Auth Bottleneck?	nein — Auth-API p95 48 ms (50 VUs) → 40 ms (750 VUs). Login-/Token-Ausgabe wurde nicht belastet (keine Testkonten)
Backend/Server Bottleneck?	nein — SSR-Antworten bleiben unter Last konstant
Storage Bottleneck?	nicht beurteilbar — im Testpfad nicht belastet
Netzwerk/CDN Bottleneck?	nein — Bundle-Asset p95 32 ms → 27 ms; Grenze lag eher beim Lastgenerator als am Edge
Nutzen der Caches/Batches	hoch für SSR (79 % → 92 % HIT). View-/Translation-Batch: nicht messbar

16./17. Empfehlungen und Prioritätenplan

Interpretation und Empfehlung — nicht Messung. Auftragsgemäß nicht umgesetzt.

Prio	Maßnahme	Problem / Nachweis	Nutzen	Risiko	Aufwand	vor Wachstum nötig?
□	Serverkennzahlen wieder verfügbar machen	/api/public/cache-metrics antwortet nach gültiger Autorisierung mit HTTP 500; DB-Health nicht abrufbar → CPU, RAM, Connections, Event-Loop-Lag, Batch-Zähler sind blind	Sättigungsgrenze wird überhaupt bestimmbar	gering (nur Messpfad)	klein	ja
□	Dedizierte Production-Testkonten (nur lesend, isoliert)	Angemeldete Pfade inkl. View-/Translation-Batch, Messenger, Market-Daten sind ungetestet	Lasttest deckt die tatsächlich teuren Pfade ab	mittel (Prod-Daten) → strikt getrenntes Konto	klein	ja
□	CDN-Caching für öffentliche HTML-Seiten aktivieren (separat bewertet)	HTML trägt cache-control: no-cache; der s-maxage-Zweig greift nicht, weil bereits eine cache-control-Kopfzeile gesetzt ist. SSR-Cache ist instanzlokal (sequenzielle Aufrufe: „miss“)	öffentliche Seiten würden am Edge statt in der Instanz bedient; entlastet Worker und DB deutlich stärker als heute	mittel — Fehlkonfiguration könnte personalisierte Inhalte cachen; vary: Cookie, Authorization ist bereits gesetzt	mittel	empfohlen vor 5.000+
□	Lastprofil mit Medien/Storage erweitern	Storage im Test nicht belastet, obwohl Feed-Medien der größte Bandbreitenposten sind	realistische Bandbreiten- und Storage-Größen	gering	klein	nein

Prio	Maßnahme	Problem / Nachweis	Nutzen	Risiko	Aufwand	vor Wachstum nötig?
□	Lasttest von mehreren Regionen	Messung nur aus einer Edge-Region (CDG)	Erkennt regionale Latenz- und Cache-Effekte	gering	klein	nein
□	Compute-Instanz vergrößern	Kein Sättigungsnachweis bei 750 VUs	aktuell kein messbarer Nutzen	–	–	nein
□	Index-/Query-Optimierung	Keine Latenzsteigerung im Lese-pfad messbar	aktuell kein messbarer Nutzen	–	–	nein

Ausbaustufen — ausschließlich als Ableitung aus dem Gemessenen, ohne Kapazitätsversprechen: 1.000 Nutzer: keine Maßnahme aus der Messung ableitbar; zuerst Beobachtbarkeit herstellen (□). 5.000 Nutzer: nicht gemessen; erforderlich sind Edge-Caching öffentlicher Seiten und ein Lasttest der angemeldeten Pfade, da genau diese ungemessen sind. 10.000 Nutzer: nicht gemessen; ohne DB-/Connection-Kennzahlen ist keine belastbare Aussage möglich — Voraussetzung ist eine Messung mit angemeldeter Schreib-/Leselast. 100.000 Nutzer: aus dieser Messung nicht ableitbar; grundsätzlich betrifft das Architekturebenen (Read-Replicas, dedizierte Medien-/CDN-Strategie, Warteschlangen für Schreibpfade), die hier bewusst nicht quantifiziert werden.

18. Abbruchgrund

Entfällt — kein Abbruch. Kein Abbruchkriterium wurde ausgelöst (keine 5xx-Häufung, keine Timeout-Welle, keine Auth-Instabilität, keine Rate-Limit-/Abuse-Auslösung, kein Rückstau).

19. Fazit

Die Production-Version von Y-Dude verhält sich bis 750 gleichzeitigen anonymen Lesenutzern (366 Requests/s, 49.952 Requests in 120 s) fehlerfrei und mit konstanter Latenz. Der SSR-Kurzzeitcache ist der wirksame Skalierungsfaktor und arbeitet sicherheitskonform (keine Vermischung personalisierter Anfragen). Nicht abgedeckt bleiben angemeldete Pfade und serverseitige Kennzahlen; beides ist Voraussetzung, um eine echte Obergrenze zu bestimmen. Am Production-System wurde nichts verändert.

20. Getestete Maximalstufe

750 virtuelle Nutzer erfolgreich getestet — Stufe 5 vollständig durchlaufen, 0 Serverfehler, 1 Timeout (0,002 %), p95 45 ms.

Rohdaten (maschinenlesbar): docs/Y-DUDE_PRODUCTION_LOADTEST_750_USERS_2026-08-30.json — enthält je Stufe alle Zähler, Statuscodes und Perzentile pro Nutzeraktion.